A Methodology to Select and Adjust Quantum Noise Models Through Emulators: Benchmarking Against Real Backends

J.A. Bravo-Montes

andres.bravo@eviden.com

Complutense University of Madrid

Miriam Bastante

Atos (Spain)

Guillermo Botella

Complutense University of Madrid

Alberto del Barrio

Complutense University of Madrid

F. García-Herrero

Complutense University of Madrid

Research Article

Keywords: NISQ era, Quantum Computing, Quantum Noise Model, Quantum Emulator, IBM-QPU, Qiskit, Qaptiva

Posted Date: July 3rd, 2024

DOI: https://doi.org/10.21203/rs.3.rs-4575191/v1

Additional Declarations: No competing interests reported.

Version of Record: A version of this preprint was published at EPJ Quantum Technology on October 22nd, 2024. See the published version at https://doi.org/10.1140/epjqt/s40507-024-00284-4.

A Methodology to Select and Adjust Quantum Noise Models Through Emulators: Benchmarking Against Real Backends

J.A. Bravo-Montes[1,2*], Miriam Bastante[1*], Guillermo Botella[3†], Alberto del Barrio[3†], F. García-Herrero[3†]

[1*]HPC & Quantum, Eviden Iberia, Madrid, 28037, Spain.
[2]Faculty of Informatics, Complutense University of Madrid, Madrid, 28040, Spain.
[3]Department of Computer Architecture and Automation, Complutense University of Madrid, Madrid, 28040, Spain.

*Corresponding author(s). E-mail(s): andres.bravo@eviden.com; miriam.bastantec@eviden.com;
Contributing authors: gbotella@ucm.es; abarriog@ucm.es; francg18@ucm.es;
†These authors contributed equally to the correction of this work.

Abstract

Currently, access to quantum processors is costly in terms of time, and power. There are quantum simulators and emulators on the market that offer alternatives for evaluating the behavior of a real quantum processor. However, these emulation environments present accuracy deviations from real devices, mainly because of difficult-to-model error sources. In this study, a methodology is proposed that allows the selection of noise models and adjustment of their parameters, considering the nature of the backends (technology, topology, vendor, model, etc.). The proposed methodology is illustrated using a small superconducting example based on the ***ibm_perth*** backend (seven qubits) and a comparison between the quantum emulators ***Qaptiva*** and ***Qiskit***, where six different noise models are applied, achieving a fidelity deviation of **0.686%** at best with respect to the real device.

Keywords: NISQ era, Quantum Computing, Quantum Noise Model, Quantum Emulator, IBM-QPU, Qiskit, Qaptiva.

1 Introduction

Quantum computing is of interest due to its potential ability to provide solutions to complex tasks in various fields such as cryptography [1][2], chemistry [3][4], optimization [5][6], finance [7][8], medicine [9][10], meteorology [11], and it is also expected to outperform classical processors in latency and resolution of some commercially valuable problems in specific cases [12][13]. Currently, we are in the Noisy Intermediate Scale Quantum (NISQ) era, this encompasses quantum computers with between fifty and hundreds and thousands of logical qubits, which in some cases can surpass the simulation capacity of classical supercomputers. The Quantum Processing Units (QPUs) used in current quantum computing employ superconducting transmons [14][15], trapped ions [16], photonic [17], neutral atoms [18][19], silicon [20][21] and other technologies [22][23][24][25]. However, these computers suffer from noise that affects their operations regardless of the quantum technology used [26].

The objective is for quantum computers to become fault-tolerant. However, noise, along with qubit scalability and connectivity, is currently a major obstacle. Therefore, it is important to reliably and efficiently characterise quantum noise to comprehend the behaviour of noisy quantum processors [27][26].

In this context, the use of quantum emulators based on classical devices played an essential role. Their relevance is justified by the fact that they allow the emulation of complex theoretical scenarios, which in turn makes it possible to carry out a controlled evaluation of the different manifestations of errors in a quantum computer.

Currently, there are several emulators from various manufacturers [28][29][30][31], and the main difference between these solutions is the significant disparity in fidelity and latency when emulators and real quantum devices are compared.

Consequently, a thorough assessment of quantum emulation environments needs to consider key aspects such as:

1. ***Accuracy:*** to ensure the reliability of the results obtained, emulation environments must be able to maintain high levels of accuracy in the emulation of quantum systems with noise. Therefore, this metric is linked to the fidelity of the circuit.
2. ***Latency:*** efficiency in the execution time of quantum simulators is a prerequisite to carrying out complex experiments in an agile manner and thus reducing the associated computational costs. Quantum processors are not always available, leading to long wait queues for access, which increases latency. Quantum emulators, however, allow for the replication of the QPU, enabling experimentation without waiting for access.
3. ***Software flexibility:*** emulation environments must have a flexible architecture that allows for interoperability between different quantum programming frameworks. This flexibility facilitates the adoption of different development techniques and optimizes the efficiency of the process.
4. ***Hardware flexibility:*** quantum emulators must be able to replicate and optimize a variety of quantum technology topologies. Furthermore, they can execute circuits with an increasing number of qubits and greater depth, demonstrating that they are scalable systems. This ensures their adaptability to different research and development scenarios, allowing for a wider range of possibilities to be explored.

The aim of this study is to present a methodology to replicate the behaviour of a quantum processor through a quantum emulation environment, using the implementation of noise models with the calibration parameters of the real QPU to be replicated. To carry out this process, this methodology evaluates parameters, such as flexibility (hardware and software), fidelity deviation, and execution time (latency).

The experiments for this methodology have been conducted on two different simulation environments, the simulator Qiskit [32] and the emulator Qaptiva [33], assessing the deviation between the prediction of the fidelity of the circuits with respect to the implementation on real IBM backend. This deviation ranged from 0.7% to 13.978% for Qaptiva, and from 5.4% to 72.04% for Qiskit.

The remainder of this paper is organized as follows. Section 2 provides an overview of the relevant noise sources and models, accompanied by a detailed discussion of the calibration data used in this study. Section 3 provides a comprehensive description of its development. Section 4 analyzes the experimental results of noisy emulation on the Qaptiva 804 and Qiskit quantum emulators, in contrast to the results obtained from a real IBM-QPU. Finally, we present our conclusions and potential future work that could be carried out in this area in Section 5.

2 Background

In this section, we discuss various fundamental descriptions of noise models and their sources, critical parameters employed for characterizing and calibrating NISQ devices, and a description of the two quantum emulators that are considered for comparison proposals in the rest of the document.

2.1 Noise sources

This section shows some of the main sources of noise that affect quantum systems.

- ***Gate level:*** this type of noise occurs when quantum gates were applied. Inaccuracies in the implementation of these gates lead to errors in quantum operations and reduce the quality of the results obtained [34].
- ***SPAM errors:*** State Preparation And Measurement refer to errors that arise during the preparation and measurement of states in QPUs. Preparation and measurement are error-prone processes due hardware imperfections, which can lead to inaccurate results in quantum computations. Although these errors are essentially bit-flip channel errors, they are considered separately as they stem from different aspects of the hardware and computation and also occur in a different part of the QPU [35].
- ***Idle state:***the environment in which a quantum system is situated can introduce noise due to interactions with particles, electromagnetic radiation, temperature fluctuations, among other external factors. This can lead to the decoherence of quantum states, resulting in the loss of quantum information and measurement errors. Decoherence is a significant source of noise, with some of the most common factors being relaxation or by thermal excitation and dephasing [34][36].

Interaction with the environment primarily occurs when qubits are idle; therefore, its impact depends on the duration of the idle period and the specific qubit with which the environment interferes [34].

2.2 Noise models

Noise models are mathematical representations of undesired effects or disturbances that affect the components of quantum systems. The noise models used in this study are explained below.

- ***Depolarizing Channel noise model:*** in this noise model, qubits depolarize with a probability of p, meaning that they are replaced by a completely mixed state. The qubit has a probability of $1 - p$ of not being affected by the noise [12].
- ***Lindblad model:*** this model is based on the Lindblad master equation [37] that describes the time evolution of an open system through a differential equation, providing an accurate representation of its non-unitary dynamics.
  In this work, we will consider that idle noise is weakly coupled to our system, satisfying the Born and Markov approximations [12][38].
- ***Thermal Relaxation model:*** this noise model is related to the interaction of physical qubits with their environment, highlighting two forms of interaction: thermal decoherence and qubit dephasing over time [36].

2.3 Eviden Qaptiva quantum emulator

Eviden [39] offers a range of quantum emulators under the name ***Qaptiva*** 800 [33]. The Eviden Qaptiva quantum emulator features a complete emulation environment with essential tools for programming using various quantum paradigms, such as gate-based emulation, annealing and analogue emulation. Qaptiva allows the user to optimise, compile, and emulate code in qubits, with or without noise [40].

In this work, we will focus on the QPU dedicated to the emulation and modeling of noise. This QPU allows two modes of noise emulation to be implemented: deterministic and stochastic.

Deterministic mode

The deterministic method is based on a representation of the density matrix of the system and is therefore limited to a small number of qubits. In this approach, the noisy gate applications are equivalent to manipulating the density matrix. Due to the total storage overhead of the density matrix, this method is limited by the required RAM of the Qaptiva quantum emulator. This limitation means that with Qaptiva 816, capable of emulating up to 41 logical qubits (a logical qubit is defined as noiseless qubit), and noisy operations can be performed with up to 20 physical qubits due to the computational overload in handling density matrices of larger sizes [34].

Stochastic mode

The stochastic method is based on a vector representation of the state, which allows it to be used for a larger number of qubits. However, a large number of samples are

required to converge to a reliable result. In this approach, instead of storing the density matrix, its interpretation as a probability distribution over pure states is used.

$$\rho = \sum_i p_i \left|\Psi_i\right\rangle \left\langle\Psi_i\right|, \tag{1}$$

where ρ is the density matrix, p_i is the probability associated with each of the states and $|\Psi_i\rangle$ is a pure state.

Pure states are sampled according to the sampling probability and their evolution is calculated using techniques similar to those used for a perfect simulation. In terms of memory limitations, stochastic simulation has similar restrictions to noiseless simulation. However, it implies an additional time overhead because to obtain statistically significant average values, it is necessary to collect enough statistical samples [34].

The deterministic method is preferred for circuits with few qubits, while the stochastic method may be suitable for circuits with a larger number of qubits if the number of samples is properly adjusted to avoid excessively long execution times.

2.4 AerSimulator by Qiskit

The ***AerSimulator*** incorporates noise modeling, providing simulation results that reflect realistic conditions. In particular, the AerSimulator noise emulator has been used to carry out this work. AerSimulator offers multiple simulation methods and configurable options for each method. The default behaviour of the AerSimulator backend tries to replicate the execution of a real quantum device [41]. It is important to note that the ***"noise"*** module has been used in combination with the default simulation method known as ***"automatic"***, which selects the appropriate emulation type based on the circuit and the noise model used. The emulation options available in Qiskit's AerSimulator are as follows [42].

- ***statevector:*** enables emulation of quantum circuits and obtaining the resulting state vector.
- ***density_matrix:*** allows sampling of measurement outcomes in noisy circuits, providing information about all measurements at the end of the circuit.
- ***stabilizer:*** enables emulation of noisy Clifford circuits, as long as the noise model errors are also Clifford errors.
- ***extended_stabilizer:*** provides an approximate emulation for circuits based on a decomposition of the state into a classified stabilizer state. The number of terms increases with the number of non-Clifford gates.
- ***superop:*** emulates the superoperator matrix of the circuit itself rather than the evolution of an initial quantum state but does not support measurement.

3 Proposal

In this section, we propose a methodology to achieve high accuracy in emulation environments with respect to a defined QPU by implementing noise models calibrated with the real values of a quantum processor. This methodology is agnostic, both in the use of processors and in emulation environments, thus allowing the comparison

of the accuracy of different emulation environments and noise models against one or several quantum processors.

Figure 1 illustrates the high-level methodology process, which consists of iterating until the $\delta Noise$ reaches the established accuracy threshold. Once the most accurate noise models are identified, the hardware noise model is loaded into the emulation environment, and quantum circuits or algorithms are run, reproducing the behavior of a noisy quantum processor.

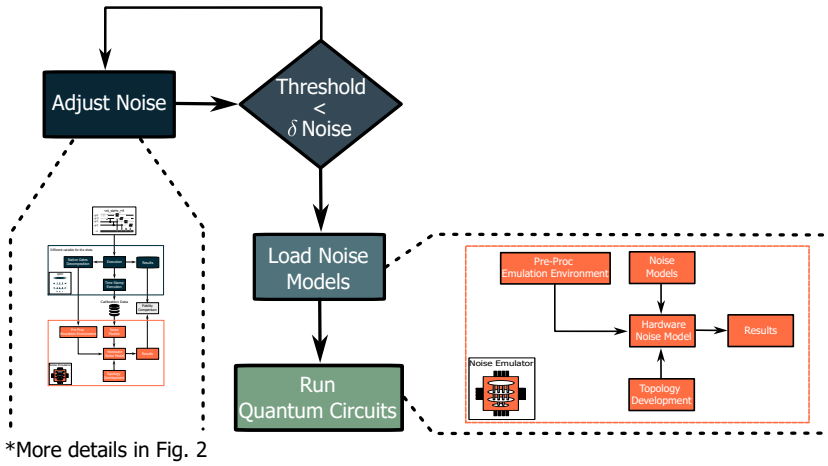

Fig. 1 Tuning for adjusting noise models in a quantum emulation environment.

The following is a detailed description of the tuning stage corresponding to the ***Adjust Noise*** module of the proposed methodology.

3.1 Proposal for Noise Adjustment Methodology

This section details the adjustment process of the noise models, the objective of which is to compare the accuracy deviation between the evaluated emulation environment and quantum processor.

Figure 2 illustrates the methodology proposed in this study. As can be seen, this alternative consists of two modules. The first module corresponds to the quantum processor environment and is used to obtain the reference metrics and calibration parameters. The second module corresponds to the simulation environment. It is important to mention that, once the noise model that obtains the highest accuracy with respect to the reference QPU is identified, the noise adjustment process concludes. Subsequently, the second orange-colored module is implemented in the ***Load Noise Models*** and ***Run Quantum Circuits*** modules in Figure 1.

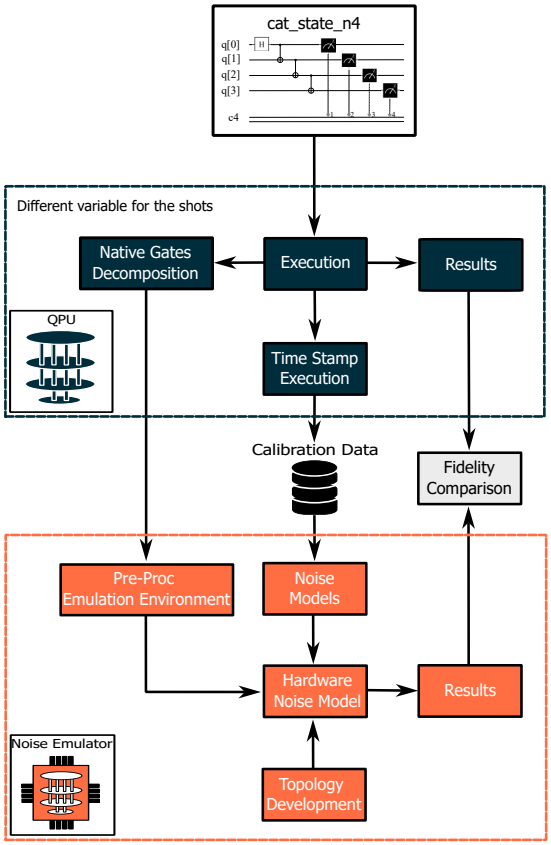

Fig. 2 Methodology proposal to adjust noise accuracy.

The metrics evaluated in the proposed methodology for emulation environments were software and hardware flexibility, reproducibility of noise models, fidelity deviation, and execution time (latency).

3.1.1 Workflow

The proposed methodology starts after the selection of a quantum circuit to be evaluated; then, this circuit is transpiled and executed in the QPU environment to obtain the calibration parameters (i.e., Table 1) and reference metrics. To create the hardware noise model in the emulation environment, it is necessary to perform the following steps: i) adapt the circuit to the emulation environment; ii) emulate and optimize the topology of the reference backend; and iii) implement the noise models. Finally, an analysis of the metrics obtained from both environments was performed for comparison.

Next, we describe in detail how the reference metrics were obtained using the proposed methodology for emulation environments.

Table 1 Calibration data of IBM-QPU.

Properties	Parameters	Units
Qubits	$T1$	μs
	$T2$	μs
	readout error	-
	prob meas0 prep1	-
	prob meas1 prep0	-
	readout length	ns
Quantum gates	gate error	-
ID, RZ, SX, X, CX	gate length	ns

3.2 Metrics evaluated in the proposed methodology

To illustrate the results obtained with this methodology, a small backend (***ibm_perth***) was selected to more easily analyze the impact of the proposal. The noise emulation environments Qaptiva and Qiskit were used as the reference backends.

This proposal allows the implementation of different emulation environments, depending on the specific needs of the user. Notably, each proposed step can be successfully adapted to each emulation environment because the conceptual basis remains unchanged. Each evaluation parameter is described in detail below.

3.2.1 Software flexibility

At this stage, the integration process of the circuits evaluated in the emulation environment occurs, and this process varies depending on the environment. Because this procedure guarantees the accuracy of the metrics to be evaluated, this methodology proposes that circuit integration is performed after the circuit execution stage in the QPU module.

Pre-processing for Qaptiva

In order to integrate the evaluated circuits into the Qaptiva environment, the interoperability module [43] has been used. This method acts as a bridge that allows the efficient and accurate conversion of quantum circuits designed in different quantum programming languages to the Qaptiva environment and vice versa.

The frameworks supported by this solution are: Qiskit (IBM) [44], OpenQASM (Qiskit, Cirq, ProjectQ, etc.)[45], PyQuil (Rigetti) [46], ProjectQ (PennyLane[47], IonQ[48]) [49][50], and Cirq (Google) [51].

Pre-processing for Qiskit

The circuits evaluated in the Qiskit environment were integrated directly, given that the evaluated QPU used the same programming language as the emulation environment. However, currently there are plugins enabling the use of Qiskit from other emulation frameworks such as: Alice & Bob [52], PennyLane [47], IonQ [48], and QRydDemo [53].

3.2.2 Hardware flexibility

In this stage, an accurate representation of the backend connectivity is developed, allowing to describe the physical layout, ensuring the accurate emulation of QPU conditions.

During this process, techniques are employed to add SWAP gates to the circuit, in the case of superconducting technology, facilitating the exchange of information between unconnected qubits. In addition, at this stage, it is necessary to establish the initialization order of the qubits in the quantum circuit.

Qaptiva topology

Qaptiva includes a plugin called ***Nnizer*** [54], which allows the correct implementation of the topology of a backend according to the coupling map of the qubits it presents.

Nnizer allows the topology of any provider using two specific configurations. The first corresponds to the selection of the SWAP gate insertion method and offers four available algorithms: ***atos*** [55], ***sabre*** [56], ***bka*** [57], and ***pbn*** [58]. The second configuration allows implementation of the initial mapping of the qubits and offers the following configurations: ***annealing***, ***reverse***, and ***none***. Both ***Nnizer*** configuration methods are explained in more detail in Appendix B.

In Table 2, a brief analysis of the implementation of ***Nnizer*** in the circuits evaluated in this experiment is shown. It should be noted that the ***pbn*** method does not include gate counting because this method does not support the use of custom gates, such as the ***ibm_perth*** backend's own ***SX*** gate.

For this work, we chose the ***sabre method*** and the initial ***none*** owing to the lower number of SWAP gates introduced in the circuits.

Table 2 ***Nnizer*** plugin evaluation.

Backend	Method	Initial mapping	# Gates Circuit 1[1]	# Gates Circuit 2[2]	# Gates Circuit 3[3]	# Gates Circuit 4[4]	# Gates Circuit 5[5]
ibm_perth	atos	annealing	42	26	26	261	20
	atos	reverse	45	26	26	261	21
	atos	none	46	26	26	261	21
	sabre	annealing	43	26	26	261	20
	sabre	reverse	43	26	26	261	21
	sabre	none	43	26	26	261	21
	bka	annealing	43	26	26	261	20
	bka	reverse	43	26	26	261	21
	bka	none	48	26	26	261	21
	pbn	annealing	-	-	-	-	-
	pbn	reverse	-	-	-	-	-
	pbn	none	-	-	-	-	-
	Initial Circuit		40	26	26	261	20

[1] Circuit 1: adder_n4; Configuration 111.
[2] Circuit 2: iswap_n2; Configuration 11.
[3] Circuit 3: grover_n2; Configuration 11.
[4] Circuit 4: error_correctiond3_n5; Configuration 00000.
[5] Circuit 5: cat_state_n4; Configuration 111.

Qiskit topology

To implement the topology in Qiskit, we first need to obtain information about the coupling map of the backend we want to emulate. If the backend belongs to IBM, we can utilize the Fake provider [59], a module containing simulated backend classes, to obtain the topology of the selected backend. In this study, we used **FakePerth**, which provides the topology of the Perth backend that we need. However, if we wish to implement a topology that is not available in IBM environments, we need to know the coupling map to incorporate it in a similar manner to that in Qaptiva.

3.2.3 Reproducibility of noise models

In this study, six combinations of different noise models were explored, equally distributed between the Qaptiva and Qiskit emulation environments. The list of noise models evaluated covers a wide variety of effects that can arise during the implementation of quantum circuits, including qubit errors due to decoherence or relaxation, initialization and measurement errors (SPAM), one and two qubit gate errors, and idle errors caused by external factors such as thermal fluctuations or electromagnetic interference.

Qaptiva hardware noise models

The noisy QPU of Qaptiva offers two modes of noise emulation: deterministic and stochastic, as explained in detail in Section 2.3. The main difference between both modes lies in the approach to quantum noise in the simulation. It is important to note that the deterministic mode follows a precise path by considering the complete density matrix, whereas the stochastic mode opts for stochastic sampling with associated statistical uncertainty, sacrificing some of this precision to reduce computational cost. Each noise model detailed below was implemented in both deterministic and stochastic modes.

1.– ***Depolarizing Channel noise model.*** The application of the Depolarizing Channel noise model in Qaptiva environment starts with the specification of channels corresponding to each gate type. In this study, a four-component multichannel has been used, based on Pauli depolarization with its associated matrices.

- ***The first noise source of error*** focuses on the preparation of quantum states, which are included in SPAM errors. In this context, a special gate called ***"initialization"*** has been designed for its representation. By implementing this gate, it is possible to quantify the probabilities associated with the erroneous preparation of the states of a qubit. To simulate this type of noise, the calibration parameter ***"error gate"*** of the **identity gate** has been used.
- ***The second noise source of error*** focuses on the study of depolarization affecting gates operating on a single qubit. For the application of errors in each of these gates, the values corresponding to the calibration parameters have been considered.
- ***The third noise source of error*** evaluates the probability of depolarization in operations involving multi-qubit gates. For instance, in the case of the CNOT gate, which is a multiqubit gate, to apply the corresponding error, the value associated with the respective calibration parameter has been considered.

- ***The fourth noise source of error*** focuses on the probability of depolarization due to measurements performed in a quantum circuit, that is, it is associated with another type of SPAM error.

Figure 3 presents a diagram illustrating the integration of the noise sources used for this model through an example. In this scenario, we chose the *cat_state_n4* circuit due to its simplicity.

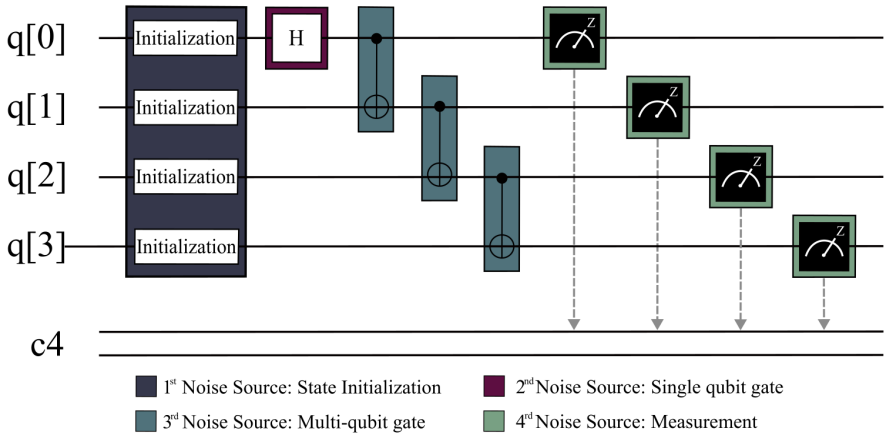

Fig. 3 Noise sources of the *cat_state_n4* circuit, where boxes representing SPAM noise (initialization and measurement) and gate noise (single-qubit and multi-qubit) are shown.

2.– ***Lindblad model.*** To apply the Lindblad model in the Qaptiva environment, it is necessary to begin by creating a hardware model that is included within a Qaptiva class. In this model, the gates, gate application times, ***"gate length"*** , and noise applied to the idle qubits must be specified. In particular, amplitude damping and pure dephasing channels have been considered for the idle qubits.

Figure 4 illustrates an example of how the Lindblad model has been implemented, using a simple circuit. As can be seen, this model mainly impacts the idle qubits.

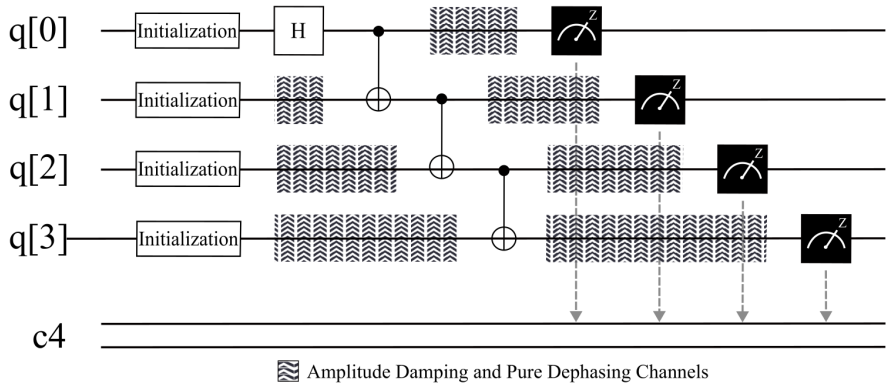

Fig. 4 Idle noise represented in the *cat_state_n4* circuit where amplitude damping and phase damping are included.

3.– ***Depolarizing Channel noise model+Lindblad model.*** For the simultaneous implementation of the Depolarizing Channel and the Lindblad noise models in the Qaptiva environment, the previously outlined steps are employed for each noise model. Initially, the noise channels specified by the Depolarizing Channel noise model are allocated to each group of quantum gates. Following that, the hardware model is established, specifying the features of the selected quantum device. This encompasses the integration of idle noise for inactive qubits through the utilization of the amplitude damping and pure dephasing channels, corresponding to the Lindblad model. In Figure 5, the circuit *cat_state_n4* is depicted, illustrating both the noise sources from the Depolarizing Channel noise model and the idle noise due to the Lindblad model.

The joint implementation of both noise models is particularly interesting as it represents a more comprehensive approach to noise modeling, encompassing both the noise sources related to gates and qubits, as well as idle noise. It is noteworthy that this implementation has been developed entirely for the purpose of conducting the present study.

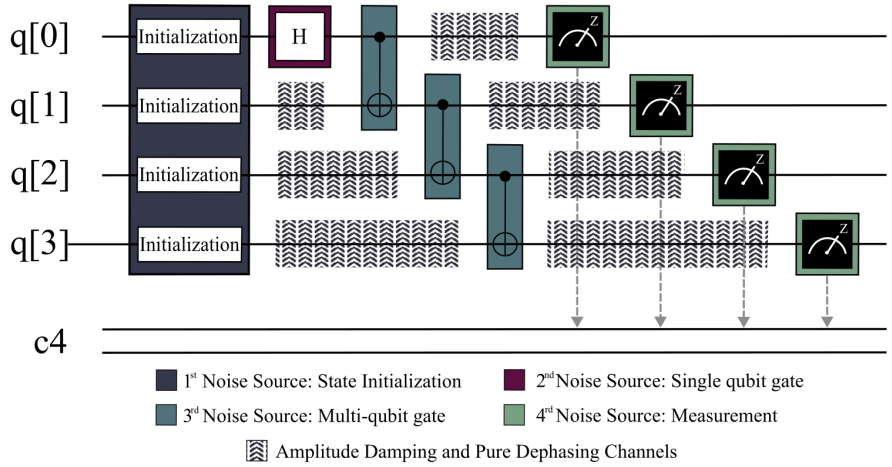

Fig. 5 Noise sources and idle noise of the *cat_state_n4* circuit.

Qiskit hardware noise models

The quantum emulator AerSimulator in Qiskit offers different emulation modes based on the circuit used and the applied noise model. The default emulation method ***"automatic"*** has been chosen, which selects the emulation type from those listed in Section 2.4 depending on the circuit and the noise model used [42]. Since the noise models and the type of gates in the circuits exhibit similar characteristics, the same emulation method has been applied in all iterations. After selecting the applied emulation method, the detailed noise models below will be implemented.

4.– ***Depolarizing Channel noise model.*** This noise model implemented in Qiskit resembles that of the Qaptiva environment but requires a specific explanation due to the change in framework. Firstly, the noise model is created and the Depolarizing Channel is applied. Similar to Section 3.2.3, four sources of noise are established: quantum state preparation, single-qubit gates, multi-qubit gates, and measurements performed. The definition of the Depolarizing Channel in Qiskit differs from that in Qaptiva. In Qiskit, it is based on the trace of the density matrix(ρ) multiplied by the identity matrix, whereas in Qaptiva, it is based on the Pauli matrices.

5.– ***Thermal Relaxation model.*** The ***"thermal_relaxation_error"*** noise model included in Qiskit is equivalent to the Lindblad model used in Qaptiva. This error model is based on the thermal relaxation of the qubits in the environment, where each qubit is parameterized with the constant value of the thermal relaxation T_1 and the constant value of the decoherence time T_2, subject to the condition $T_2 \leq 2T_1$. Additionally, the application time of each gate, known as ***"gate length"***, must be specified.

6.– ***Depolarizing Channel noise model+Thermal Relaxation model.*** After explaining the separate implementation of the Depolarizing Channel and Thermal

Relaxation noise models in Qiskit, they are jointly implemented similarly to the implementation in Qaptiva. Firstly, we apply the noise model due to the depolarization of the channel associated with each type of gate. Then, we apply the noise due to inactive qubits by specifying T_1, T_2, and the application time of each gate.

3.2.4 Fidelity and latency

This experiment was characterized by an accurate evaluation of the fidelity and latency metrics.

The methodology needs to be suitable for various quantum circuits and algorithms with different characteristics, which may have different configurations in terms of inputs and outputs. To ensure a comparable metric across all circuits, we introduced the concept of **circuit fidelity**, also referred to as the **success rate**. Below, we will explain how this metric was obtained in different scenarios.

1. Each output was averaged for each possible input configuration of the circuit, with n possible input configurations and a single output for each configuration.
2. Circuit with n possible input configurations and n possible outputs per configuration. First, the sum of the probabilities of the n possible outputs is calculated to obtain a single fidelity for each input configuration, and then the average of each output for each possible input configuration of the circuit is calculated.

We used backend calibration data to compare the **fidelity** of the quantum circuits for each noise model implemented, to determine which model or combination of models produces the least deviation from the backend used.

The **latency metric** allowed us to evaluate the execution time of each noise model for each quantum emulator or QPU. For quantum emulators, this includes all processes in the orange module in Figure 2, including the construction and execution times of the employed noise models. For the QPU, it involves the processes in the dark blue module in Figure 2, as well as the waiting time in the queues to access the QPU. This metric enables the comparison of latency times between emulation and real backends and facilitates comparisons among different quantum emulators for various circuits and noise models.

3.3 Illustrative example of methodology application

The methodology described in this example has been applied to all noise models and circuits. However, to avoid excessive length in this paper, only the graphical representation of the *cat_state_n*4 circuit is included. The data obtained regarding all noise models of all circuits are reflected in the fidelity and latency metrics presented in Table 4 and Table 6.

Figure 6 depicts the *cat_state_n*4 circuit, showcasing its eight possible configurations. This circuit has 8 possible inputs and 16 possible outputs, with the outputs represented on the horizontal axis. The difference between the number of inputs and outputs is due to the circuit's structure, which requires four qubits: three input qubits and one ancilla qubit. The values used to create Figure 6 are included in Tables D2 and D3 for Qaptiva, and in Tables D4 and D5 for Qiskit, which are included in appendix D.

These figures show the implementation of the three noise models: Depolarizing Channel, Lindblad/Thermal Relaxation model, and Depolarizing Channel+Lindblad/Thermal Relaxation model in the Qaptiva and Qiskit environments. Each emulator has implemented this process using the calibration parameters obtained by backend ***ibm_perth*** for each execution. This ensures that each execution is comparable, as calibration parameters vary.

In all possible circuit configurations, the model with the highest deviation in fidelity from the ***ibm_perth*** backend is the Lindblad/Thermal Relaxation model, followed by the Depolarizing Channel model. This work proposes unifying the two previous noise models to examine their probability profiles. This combined model demonstrates less deviation in fidelity concerning the backend used compared to the models separately in both the Qaptiva and Qiskit environments.

In all the models, we observe two peaks of maximum probability, which indicate the outputs of these states, i.e., the states that the circuit returns after a given input. It is apparent that each input configuration leads to different final states, and for the cat_state_n4 circuit, the output always results in two states. In all configurations, when we examine the graph corresponding to Lindblad/Thermal models + Depolarizing Channel, all states show a probability more similar to the reference backend in both Qaptiva and Qiskit, with the greatest difference observed in the peaks of maximum probability. Additionally, we observe worse correspondences of noise modeling in the Qiskit environment, deviating from the ***ibm_perth*** backend probability profile.

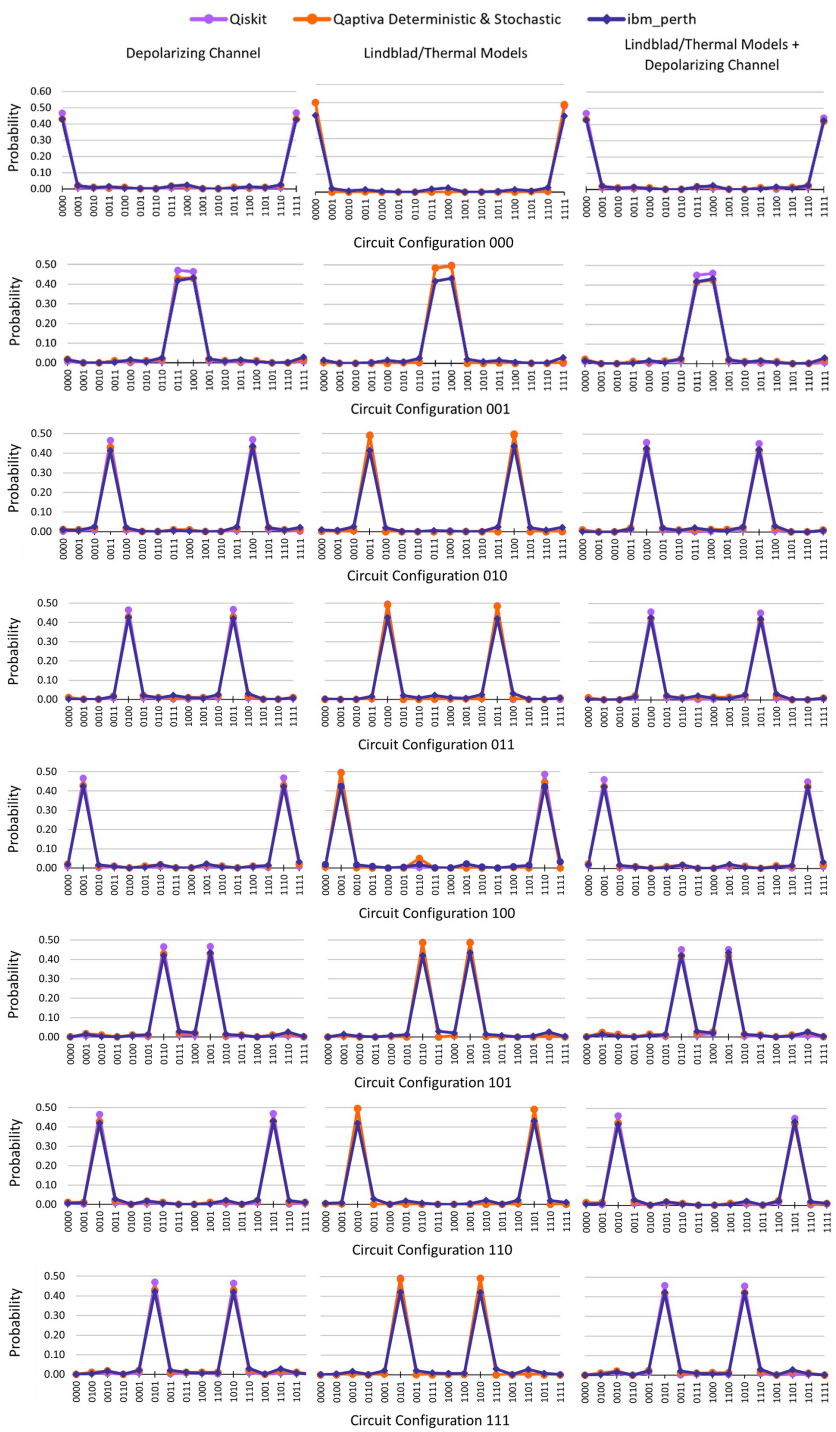

Fig. 6 Distribution of probability peaks for the three noise models in the *cat_state_n*4 circuit for all possible configurations: $000, 001, 010, 011, 100, 101, 110, 111$.

4 Results

The results section is divided into two parts, providing a complete overview of the methodology proposed in Figure 2. The first approach corresponds to the adjustment phase, in which various noise models are evaluated using the selected circuits in the proposed emulation environment. The second approach focuses on a comparative analysis between the Qaptiva and Qiskit emulation environments, evaluating the fidelity deviation and execution time (latency) compared with the ***ibm_perth*** reference backend.

Quantum circuits have been selected by evaluating different characteristics, such as the width, depth, number of gates of one qubit, number of gates of two qubits, and functionality. Table 3 lists the circuits used in this study together with their corresponding characteristics.

Table 3 Characteristics of circuits.

Circuit Name	Circuit Width	Circuit Depth	Quantum Gates	Functionality
adder_n4[60]	4	12	23	Quantum Arithmetic
cat_state_n4[61]	4	5	4	Logical Operation
grover_n2[62]	2	12	16	Grover's Algorithm
iswap_n2[63]	2	8	9	Logical Operation
error_correctiond3_n5[64]	5	78	114	Error Correction

To ensure that the evaluated metrics provide a robust benchmark, the criteria set out in Equation 2 were applied to determine the number of runs in each quantum circuit.

$$Total_{exec} = 10\ runs \times circuit\_configuration. \tag{2}$$

The number of runs for each circuit evaluated, with 8192 shots per run, is shown below.

- $adder\_n4$: 80 executions.
- cat_state_n4: 80 executions.
- $grover\_n2$: 40 executions.
- $iswap\_n2$: 40 executions.
- $error\_correctiond3\_n5$: 10 executions.

4.1 Adjust noise models

This section presents the results obtained after applying the noise models to each emulation environment for each evaluated circuit. It also compares the six noise models implemented and the ***ibm_perth*** backend. Our goal is to select the noise model that best fits the backend for each environment. For this purpose, we have used the relative percentage error (ε_r), defined as the ratio of the absolute error to the actual value of the magnitude, expressed as a percentage.

The six noise models that have been evaluated are the following:

1. Depolarizing Channel
2. Lindblad Model
3. Depolarizing Channel+Lindblad Model

} ***Qaptiva Environment***

4. Depolarizing Channel
5. Thermal Relaxation Model
6. Depolarizing Channel+Thermal Relaxation Model

} ***Qiskit Environment***

The relative percentage error between the backend used and the noise models employed in both Qaptiva and Qiskit is shown in Table 5. This error was calculated using Equation 3, which determines how closely the emulator values match the actual backend values [65].

$$\varepsilon_r(\%) = \frac{|V_1 - V_2|}{V_1} \times 100. \tag{3}$$

Where "V_1" corresponds to the fidelity values obtained from the ***ibm_perth***, while "V_2" refers to the fidelity values obtained for all noise models implemented in both Qaptiva and Qiskit.

4.1.1 Fidelity results

This subsection presents the fidelity results of the circuits evaluated both in the ***Qaptiva*** environment and in the deterministic and stochastic modes, as well as ***Qiskit*** and the ***ibm_perth*** backend. As shown in Table 4, a single benchmark metric was obtained for each circuit following the success rate criterion mentioned above.

The fidelity values obtained from the Depolarizing Channel noise model are slightly higher than those obtained from the QPU, but they are surpassed by those of the Lindblad or Thermal Relaxation noise model, which exhibit values close to unity in most circuits. This is because the model affecting idle noise generates less interference than that generated by the Depolarizing Channel model. Therefore, when applying the two noise models together, a lower fidelity value is observed than that obtained by the Depolarizing Channel model, which is closer to that obtained by the QPU.

If we compare the fidelity of the circuits implemented with the noise model that most closely approximates the value of the QPU, we observe that the *grover_n2* circuit shows the highest fidelity, with values ranging from 0.905 to 0.941, followed by the *iswap_n2* circuit, with values ranging from 0.904 to 0.942. On the other hand, the *error_correctiond3_n5* circuit shows the lowest fidelity, with values ranging from 0.5 to 1. The *error_correctiond3_n5* circuit indicating no variation among the obtained values. This is mainly because there are many valid outputs, which results in the fidelity value not showing significant variations. These variations are mitigated by utilizing a metric that relies on a single value.

Table 4 Fidelity results of Qaptiva and Qiskit with its corresponding standard deviation.

Fidelity Results				
	QPU	**Qaptiva: Deterministic Mode**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Lindblad Model**	**Depolarizing Channel+ Lindblad Model**
adder_n4	0.66 ± 0.04	0.717 ± 0.012	0.95 ± 0.04	0.689 ± 0.017
cat_state_n4	0.848 ± 0.004	0.8578 ± 0.0006	0.957 ± 0.016	0.843 ± 0.006
grover_n2	0.89 ± 0.03	0.9998 ± 0.0001	0.9053 ± 0.0015	0.9053 ± 0.0015
iswap_n2	0.89 ± 0.03	0.9041 ± 0.0001	0.9998 ± 0.0002	0.9039 ± 0.0003
error_correctiond3_n5	0.58 ± 0.08	0.375 ± 0.0	0.5 ± 0.0	0.5 ± 0.0
	QPU	**Qaptiva: Stochastic Mode**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Lindblad Model**	**Depolarizing Channel+ Lindblad Model**
adder_n4	0.66 ± 0.04	0.717 ± 0.012	0.959 ± 0.016	0.689 ± 0.017
cat_state_n4	0.848 ± 0.004	0.8577 ± 0.0006	0.982 ± 0.007	0.842 ± 0.006
grover_n2	0.89 ± 0.03	0.9054 ± 0.0014	0.9996 ± 0.0001	0.9053 ± 0.0017
iswap_n2	0.89 ± 0.03	0.9039 ± 0.0004	0.9996 ± 0.0002	0.9039 ± 0.0004
error_correctiond3_n5	0.58 ± 0.08	0.375 ± 0.0	0.5 ± 0.0	0.5 ± 0.0
	QPU	**Qiskit**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Thermal Model**	**Depolarizing Channel+ Thermal Model**
adder_n4	0.66 ± 0.04	0.810 ± 0.012	0.93 ± 0.02	0.74 ± 0.04
cat_state_n4	0.848 ± 0.004	0.9336 ± 0.0007	0.983 ± 0.002	0.909 ± 0.003
grover_n2	0.89 ± 0.03	0.9598 ± 0.0018	0.985 ± 0.004	0.941 ± 0.008
iswap_n2	0.89 ± 0.03	0.96 ± 0.03	0.986 ± 0.006	0.94 ± 0.03
error_correctiond3_n5	0.58 ± 0.08	1.0 ± 0.0	1.0 ± 0.0	1.0 ± 0.0

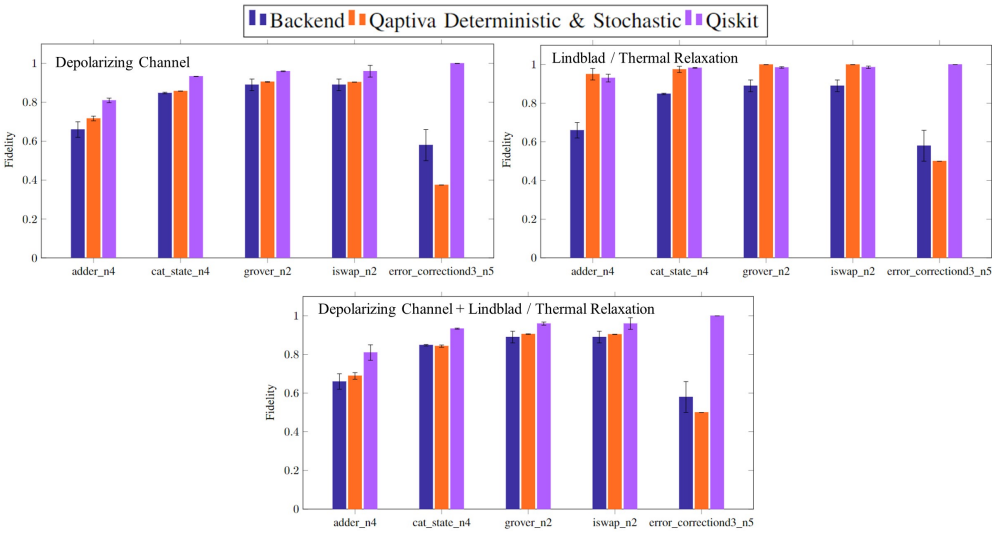

Fig. 7 Comparison of fidelity across different frameworks used and various models.

In Figure 7, a graphical representation of the values shown in Table 4 is displayed. This bar chart visualizes the fidelity difference between the backend and the implemented noise models, including the standard deviations represented with error bars.

The fidelity values used as the basis for this relative percentage error are found in Table 7. These values are the average of all executions performed for each circuit, including different configurations.

The percentage standard deviations, along with the relative percentage fidelity errors for each noise model and circuit, are detailed in Table 5. It is important to note that two evaluation methods were used: deterministic and stochastic. Both methods returned values that fall within the standard deviation range for each model and circuit. For this reason, the values reflected in Table 5 correspond exclusively to the deterministic method.

Once the comparison between the noise models is completed, it is observed that the combined Lindblad/Thermal Relaxation models+Depolarizing Channel model presents values closer to those of the backend used. For this reason, these selected noise models will be implemented in the next section, following the proposed methodology.

4.1.2 Latency results

In this subsection, the latency results shown in Table 6 for the noise models applied to the circuits implemented in the emulators, Qaptiva and Qiskit, as well as in the ***ibm_perth*** backend, are presented and explained.

On the one hand, when analyzing the different models implemented in Qaptiva, it is observed that the more complex model, which combines the Depolarizing Channel and Lindblad models, exhibits higher latency than the individual models. On the

Table 5 Relative percentage error of fidelity of Qaptiva and Qiskit with its corresponding standard deviation.

Relative percentage error of fidelity			
Qaptiva: Deterministic & Stochastic Modes			
Circuit	**Depolarizing Channel %**	**Lindblad Model %**	**Depolarizing Channel + Lindblad Model %**
adder_n4	8.6 ± 1.2	44 ± 3	4.4 ± 1.7
cat_state_n4	1.11 ± 0.06	15.0 ± 1.6	0.7 ± 0.6
grover_n2	1.45 ± 0.15	12.031 ± 0.012	1.44 ± 0.15
iswap_n2	1.441 ± 0.015	12.19 ± 0.02	1.43 ± 0.03
error_correctiond3_n5	35.484 ± 0.0	13.978 ± 0.0	13.978 ± 0.0
Qiskit: AerSimulator			
Circuit	**Depolarizing Channel %**	**Thermal Model %**	**Depolarizing Channel + Thermal Model %**
adder_n4	22.8 ± 1.2	41 ± 2	13 ± 4
cat_state_n4	10.03 ± 0.07	15.8 ± 0.2	7.1 ± 0.3
grover_n2	7.55 ± 0.18	10.4 ± 0.4	5.4 ± 0.8
iswap_n2	8 ± 3	10.7 ± 0.6	6 ± 3
error_correctiond3_n5	72.04 ± 0.0	72.04 ± 0.0	72.04 ± 0.0

other hand, in Qiskit, the combined model shows latency times similar to those of the Depolarizing Channel and Thermal Relaxation models. This disparity between the emulators is likely due to differences in the hardware used for the application of the noise models.

When comparing the latencies of the emulators with those of the QPU, significant differences are observed. In the case of Qaptiva in the deterministic mode, there are differences are approximately four orders of magnitude smaller in most circuits, and in the stochastic mode, differences of around 3 orders of magnitude less, except in the *error_correctiond*3_*n*5 circuit, which shows slightly higher latencies. For this reason, the ***deterministic*** mode is preferred over the ***stochastic*** one, as similar fidelity results are achieved, as seen in Table 4, but with much higher latency in the stochastic method, as seen in Table 6. For Qiskit, differences of approximately 5 orders of magnitude less are observed in most circuits.

Therefore, it is concluded that quantum circuits with more qubits and higher depth have longer execution times. However, in the QPU, latency is mainly due to waiting queues and not to the circuit execution time on it.

4.2 Performance evaluation of quantum emulation environments

This section presents the results obtained after a comparative analysis between the Qaptiva and Qiskit emulation environments in terms of accuracy deviation and execution times. For this comparison, the noise models that obtained the best accuracy

Table 6 Latency results.

Latency Results (Seconds)				
	QPU	**Qaptiva: Deterministic Mode**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Lindblad Model**	**Depolarizing Channel+ Lindblad Model**
adder_n4	$0.8645 \cdot 10^4$	0.271	0.220	$0.1424 \cdot 10^1$
cat_state_n4	$0.1593 \cdot 10^4$	0.168	0.150	0.877
grover_n2	$0.1430 \cdot 10^4$	0.187	0.145	0.926
iswap_n2	$0.1760 \cdot 10^4$	0.169	0.160	0.923
error_correctiond3_n5	$0.1798 \cdot 10^4$	$0.7092 \cdot 10^1$	$0.6994 \cdot 10^1$	$0.7165 \cdot 10^1$
	QPU	**Qaptiva: Stochastic Mode**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Lindblad Model**	**Depolarizing Channel+ Lindblad Model**
adder_n4	$0.8645 \cdot 10^4$	$0.419 \cdot 10^1$	$0.780 \cdot 10^1$	$0.785 \cdot 10^1$
cat_state_n4	$0.1593 \cdot 10^4$	$0.6313 \cdot 10^1$	$0.3499 \cdot 10^1$	$0.122 \cdot 10^2$
grover_n2	$0.1430 \cdot 10^4$	$0.6373 \cdot 10^1$	$0.3765 \cdot 10^1$	$0.1446 \cdot 10^1$
iswap_n2	$0.1760 \cdot 10^4$	$0.8603 \cdot 10^1$	$0.3826 \cdot 10^1$	$0.135 \cdot 10^1$
error_correctiond3_n5	$0.1798 \cdot 10^4$	$0.1624 \cdot 10^4$	$0.846 \cdot 10^2$	$0.2191 \cdot 10^4$
	QPU	**Qiskit**		
Circuit	**ibm_perth**	**Depolarizing Channel**	**Thermal Model**	**Depolarizing Channel+ Thermal Model**
adder_n4	$0.8645 \cdot 10^4$	0.053	0.045	0.045
cat_state_n4	$0.1593 \cdot 10^4$	0.059	0.042	0.042
grover_n2	$0.1430 \cdot 10^4$	0.035	0.046	0.046
iswap_n2	$0.1760 \cdot 10^4$	0.033	0.031	0.031
error_correctiond3_n5	$0.1798 \cdot 10^4$	0.130	0.206	0.206

with respect to the ***ibm_perth*** backend during the tuning process were used: Depolarizing Channel Noise Model+Lindblad model noise in the Qaptiva environment and Depolarizing Channel noise model+Thermal model in the Qiskit environment.

Figure 8 graphically presents the accuracy results obtained in the emulation environments. This analysis covers a general and specific perspective by evaluating each circuit for all possible configurations.

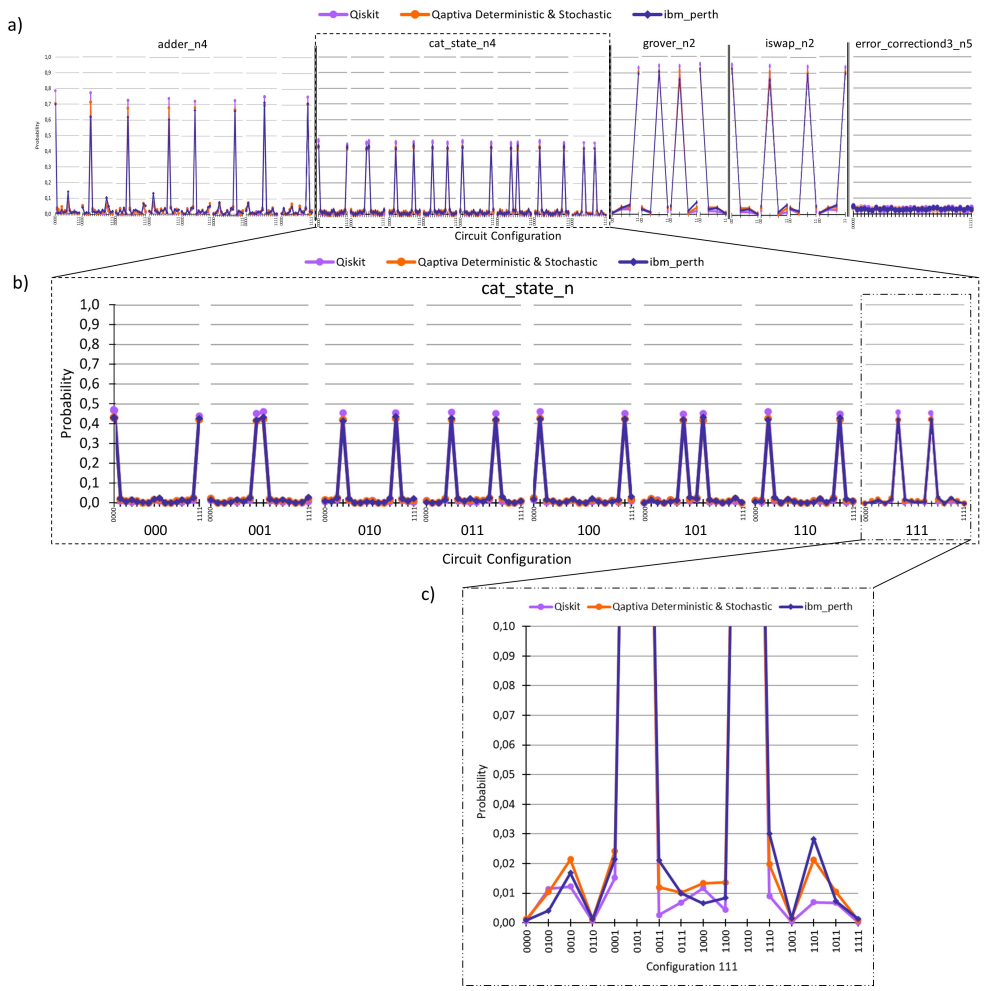

Fig. 8 Distribution of fidelity peaks in the Lindblad/Thermal Relaxation+Depolarizing Channel noise model. a. Performance representation of the evaluated circuits in all possible configurations, highlighting that the peaks with the highest probabilities indicate the output states of each circuit. b. A zoom-in on the execution of the *cat_state_n*4 circuit showing all possible circuit configurations. c. Further zoom-in shows the configuration 111 of the *cat_state_n*4 circuit, revealing in greater detail the discrepancies between the measured states due to the modeled noise among the emulators used and the backend.

In Figure 8 a), it can be observed that the circuits *adder_n*4, *grover_n*2, and *iswap_n*2 have only one output state, while the *cat_state_n*4 has two output states. The *error_correctiond*3_*n*5 circuit has 16 output states; however, because the probability of obtaining each of them is low, they are barely distinguishable from the noise induced errors in the graph. In Figure 8 b) a zoom is applied to the cat_state_n4 circuit

due to the presence of two high fidelity peaks, indicating that the circuit has two solution states. Additionally, it can be observed that these peaks vary depending on the initial circuit configuration. Both peaks show a very similar fidelity of approximately 0.43 across all configurations. In the last zoom performed on the configuration 111 of the circuit *cat_state_n*4, corresponding to Figure 8 c), it is observed that the peaks generated by the modeled noise with the Qaptiva emulator resemble more closely those from the ***ibm_perth*** backend than those obtained by the Qiskit emulator.

4.3 Quantitative comparison of the results

The selected noise model for fidelity and latency comparisons between the emulation environments and the reference backend is a combination of the Depolarizing Channel and the Lindblad model in the Qaptiva environment and the combination of the Depolarizing Channel and the Thermal Relaxation model in the Qiskit environment. This was chosen due to improvements in fidelity and deviation from the backend when unifying the noise models.

Table 7 lists the percentage deviation of the circuits evaluated in the Qaptiva (Deterministic and Stochastic) and Qiskit environments, in terms of fidelity and latency. It is important to note that the lower the fidelity percentage obtained, the better the accuracy of the modeling.

In terms of fidelity, better results were obtained in the Qaptiva environment than in the Qiskit environment, with metrics ranging from 13.978% for the *error_correctiond*3_*n*5 circuit to 0.686% for the *cat_state_n*4 circuit. Both Qaptiva modes had similar fidelity values.

Table 7 Accuracy fidelity and latency difference between Qaptiva and Qiskit emulation environments.

Percentage differences of fidelity and latency						
	Qaptiva Deterministic Mode		**Qaptiva Stochastic Mode**		**Qiskit**	
	Depolarizing Channel+ Lindblad Model		**Depolarizing Channel+ Lindblad Model**		**Depolarizing Channel+ Thermal Model**	
Circuit	**Fidelity %**	**Latency %**	**Fidelity %**	**Latency %**	**Fidelity %**	**Latency %**
adder_n4	4.436	0.016	4.410	0.091	12.948	0.0005
cat_state_n4	0.686	0.055	0.740	0.765	7.091	0.003
grover_n2	1.436	0.065	1.441	0.101	5.406	0.003
iswap_n2	1.425	0.052	1.425	0.077	5.731	0.0017
error_correctiond3_n5	13.978	0.398	13.978	121.891	72.043	0.012

In Table 7, the latency value of ***ibm_perth*** is considered as the baseline. The results obtained in both emulation environments were significantly lower than those obtained in the ***ibm_perth*** backend. The emulation times in Qaptiva's deterministic mode ranged from 0.017% to 0.4% of the backend latency, whereas in the stochastic mode, they fluctuated between 0.08% and 122%. The emulation times in the

Qiskit ranged from 0.0005% to 0.012%. The percentage of latency in the case of $error\_correctiond3\_n5$ is higher than that in other cases, especially in Qaptiva's stochastic mode, which exceeds the backend latency. This disparity is due to the larger computation time of the circuit, resulting from its depth and number of qubits used. It can be observed that latency times primarily depend on the circuit's depth, with a significant difference, especially in the case of Qiskit and Qaptiva's deterministic mode, compared to the used backend. This is mainly owed to the consideration of wait queues within the backend latency.

It is important to remark that, once again, this is just an example to illustrate the proposed methodology. However, this type of methodology can be generalized to any other layout, technology, or noise model for which the necessary information is available. Qaptiva allows for the adaptation of this methodology to gate-based technologies such as trapped ions, neutral atoms, or photonics. It provides predefined topologies like All To All, Linear, and Grid, as well as backends from IBM, Google, Rigetti, and Zuchongzhi, and the ability to create custom topologies. Furthermore, the noise models can be adjusted based on the underlying technology and the precision of the available tuning parameters.

5 Conclusions

The methodology proposed in this work allows the adjustment of the noise models and their parameters according to the available technology, data, and topology of quantum technologies such as superconductors, trapped ions, photons, neutral atoms, and silicon.

The use of quantum emulators significantly reduces execution times because of the absence of queues to execute a program, as in QPUs. Through the proposed methodology, it is possible to achieve good reproducibility of the noise models and obtain a minimum accuracy deviation from that of a real quantum processor.

Since the methodology spans various circuits, each with features such as the number of qubits or depth, it hints at the method's adaptability and suggests a potentially scalable behavior, although further validation with more cases is needed o similar. As future directions, this methodology can be applied to different quantum technologies to identify patterns in the efficiency and latency of quantum emulators, providing a deeper insight into which technology is more promising in terms of performance and scalability. Additionally, another line of research could focus on optimizing specific algorithms for various quantum architectures, developing new error mitigation techniques, and optimizing circuits, to reduce latency and improve the fidelity of quantum calculations.

6 Declarations

6.1 Ethical Approval and Consent to participate

Not applicable. Research not involving human subjects.

6.2 Consent for publication

Not applicable. Research not involving human subjects.

6.3 Availability of supporting data

All data generated or analysed during this study are included in this published article.

6.4 Competing interests

The authors declare that they have no competing interests.

6.5 Funding

This work has been supported by grant PID2021-123041OB-I00 funded by Spanish Government (Ministry of Science and Innovation).

7 List of abbreviatios

- **LNN**: Linear Nearest Neighbor.
- **NISQ**: Noisy Intermediate Scale Quantum.
- **QPU**: Quantum Processing Unit.
- **RAM**: Random Access Memory.
- **SPAM**: State Preparation And Measurement.

References

[1] Shalini Subramani, K.A. Selvi M, Svn, S.K.: Review of security methods based on classical cryptography and quantum cryptography. Cybernetics and Systems, 1–19 (2023). doi:10.1080/01969722.2023.2166261

[2] Portmann, C., Renner, R.: Security in quantum cryptography. Rev. Mod. Phys. **94**, 025008 (2022). doi:10.1103/RevModPhys.94.025008

[3] Lee, S., Lee, J., Zhai, H., Tong, Y., Dalzell, A.M., Kumar, A., Helms, P., Gray, J., Cui, Z.-H., Liu, W., Kastoryano, M., Babbush, R., Preskill, J., Reichman, D.R., Campbell, E.T., Valeev, E.F., Lin, L., Chan, G.K.-L.: Evaluating the evidence for exponential quantum advantage in ground-state quantum chemistry. Nature Communications **14**, 1952 (2023). doi:10.1038/s41467-023-37587-6

[4] Jang, S.J.: Quantum Mechanics for Chemistry. Springer, ??? (2023)

[5] Si, Y., Wang, R., Zhang, S., Zhou, W., Lin, A., Zeng, G.: Configuration optimization and energy management of hybrid energy system for marine using quantum computing. Energy **253**, 124131 (2022). doi:10.1016/j.energy.2022.124131

[6] Yarkoni, S., Raponi, E., Bäck, T., Schmitt, S.: Quantum annealing for industry applications: introduction and review. Reports on Progress in Physics **85**(10), 104001 (2022). doi:10.1088/1361-6633/ac8c54

[7] Herman, D., Googin, C., Liu, X., Sun, Y., Galda, A., Safro, I., Pistoia, M., Alexeev, Y.: Quantum computing for finance. Nature Reviews Physics **5**, 450–465 (2023). doi:10.1038/s42254-023-00603-1

[8] Sheng, Y., Li, H., Xing, T., Wei, S., Liu, Z., Zhang, J., Long, G.-L.: Bq-bank: A quantum software for finance and banking. Quantum Engineering **2023**, 7810974 (2023). doi:10.1155/2023/7810974

[9] Ur Rasool, R., Ahmad, H.F., Rafique, W., Qayyum, A., Qadir, J., Anwar, Z.: Quantum computing for healthcare: A review. Future Internet **15**(3) (2023). doi:10.3390/fi15030094

[10] Flöther, F.F.: The state of quantum computing applications in health and medicine. Research Directions: Quantum Technologies, 1–21 (2023). doi:10.1017/qut.2023.4

[11] Tennie, F., Palmer, T.N.: Quantum computers for weather and climate prediction: The good, the bad, and the noisy. Bulletin of the American Meteorological Society **104**(2), 488–500 (2023)

[12] Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information, 10th anniversary edn. Cambridge University Press, ??? (2010)

[13] Cumming, R., Thomas, T.: Using a quantum computer to solve a real-world problem – what can be achieved today? arXiv (2022)

[14] Huang, H.-L., Wu, D., Fan, D., Zhu, X.: Superconducting quantum computing: A review. Science China Information Sciences **63** (2020). doi:10.1007/s11432-020-2881-9

[15] Clarke, J., Wilhelm, F.K.: Superconducting quantum bits. Nature **453**, 1031–1042 (2008). doi:10.1038/nature07128

[16] Blatt, R., Roos, C.: Quantum simulations with trapped ions. Nature Physics **8**, 277–284 (2012). doi:10.1038/nphys2252

[17] Aspuru-Guzik, A., Walther, P.: Photonic quantum simulators. Nature Physics **8**, 285–291 (2012). doi:10.1038/nphys2253

[18] Wintersperger, K., Dommert, F., Ehmer, T., Hoursanov, A., Klepsch, J., Mauerer, W., Reuber, G., Strohm, T., Yin, M., Luber, S.: Neutral atom quantum computing hardware: performance and end-user perspective. EPJ Quantum Technology **10**(1), 32 (2023). doi:10.1140/epjqt/s40507-023-00190-1

[19] Evered, S.J., Bluvstein, D., Kalinowski, M., Ebadi, S., Manovitz, T., Zhou, H., Li, S.H., Geim, A.A., Wang, T.T., Maskara, N., Levine, H., Semeghini, G., Greiner, M., Vuletić, V., Lukin, M.D.: High-fidelity parallel entangling gates on a neutral-atom quantum computer. Nature **622**(7982), 268–272 (2023). doi:10.1038/s41586-023-06481-y

[20] Salfi, J., Mol, J., Rahman, R., Klimeck, G., Simmons, M., Hollenberg, L., Rogge, S.: Quantum simulation with dopant atoms in a semiconductor (2015)

[21] Atatüre, M., Englund, D., Vamivakas, N., Lee, S.-Y., Wrachtrup, J.: Material platforms for spin-based photonic quantum technologies. Nature Reviews Materials **3**(5), 38–51 (2018). doi:10.1038/s41578-018-0008-9

[22] Agarwal, K., Rai, H., Mondal, S.: Quantum dots: an overview of synthesis, properties, and applications. Materials Research Express **10**(6), 062001 (2023). doi:10.1088/2053-1591/acda17

[23] Altman, E., Brown, K.R., Carleo, G., Carr, L.D., Demler, E., Chin, C., DeMarco, B., Economou, S.E., Eriksson, M.A., Fu, K.-M.C., Greiner, M., Hazzard, K.R.A.,

Hulet, R.G., Kollar, A.J., Lev, B.L., Lukin, M.D., Ma, R., Mi, X., Misra, S., Monroe, C., Murch, K., Nazario, Z., Ni, K.-K., Potter, A.C., Roushan, P., Saffman, M., Schleier-Smith, M., Siddiqi, I., Simmonds, R., Singh, M., Spielman, I.B., Temme, K., Weiss, D.S., Vuckovic, J., Vuletic, V., Ye, J., Zwierlein, M.: Quantum simulators: Architectures and opportunities (2019). doi:10.1103/PRXQuantum.2.017003

[24] Rodríguez, A.R.: Moléculas ultrafrías (2015). https://www.um.es/acc/moleculas-ultrafrias/

[25] Adams, C.S., Pritchard, J.D., Shaffer, J.P.: Rydberg atom quantum technologies. Journal of Physics B: Atomic, Molecular and Optical Physics **53**(1), 012002 (2019). doi:10.1088/1361-6455/ab52ef

[26] Preskill, J.: Quantum computing in the nisq era and beyond. Quantum **2** (2018). doi:10.22331/q-2018-08-06-79

[27] Martinis, J.M.: Qubit metrology for building a fault-tolerant quantum computer. npj Quantum Information **1** (2015). doi:10.1038/npjqi.2015.5

[28] Staff, I.: The Value of Classical Quantum Simulators. [Accessed October 28, 2023] (2023)

[29] Revision, R.C.: The Quantum Virtual Machine (QVM). [Accessed October 28, 2023] (2018). https://pyquil-docs.rigetti.com/en/v2.0.0/qvm.html#

[30] IBM: IBM Quantum simulators. [Accessed October 25, 2023] (2023). https://docs.quantum-computing.ibm.com/verify/cloud-based-simulators

[31] Services, A.W.: Amazon Braket supported devices. [Accessed October 24, 2023] (2023). https://docs.aws.amazon.com/pdfs/braket/latest/developerguide/braket-developer-guide.pdf

[32] Team, Q.D.: IBM Quantum Documentation. [Accessed March 14, 2024] (2024). https://docs.quantum.ibm.com/

[33] SE, A.: Qaptiva. [Accessed October 24, 2023] (2023). https://atos.net/en/solutions/high-performance-computing-hpc/quantum-computing-qaptiva

[34] Eviden: myQLM documentation. [Accessed October 18, 2023] (2023). https://myqlm.github.io/index.html

[35] Nachman, B., Geller, M.R.: Categorizing readout error correlations on near term quantum computers (2021)

[36] Georgopoulos, K., Emary, C., Zuliani, P.: Modelling and simulating the noisy behaviour of near-term quantum computers. Phys. Rev. A **104** (2021). doi:10.1103/PhysRevA.104.062432

[37] Manzano, D.: A short introduction to the lindblad master equation. AIP Advances **10**(2) (2020). doi:10.1063/1.5115323

[38] Preskill, J.: Lecture Notes for Physics 229: Quantum Information and Computation, pp. 77–135. California Institute of Technology, ??? (1998). Chap. 3. Measurement and Evolution

[39] SAS, E.: Eviden website. [Accessed December 5, 2023] (2023). https://eviden.com/es-es/

[40] Eviden: Quantum Processing Unit. [Accessed February 20, 2024] (2024). https://myqlm.github.io/02_user_guide/02_execute/03_qpu.html#qpu

[41] Qiskit contributors: Qiskit: An Open-source Framework for Quantum Computing (2023). doi:10.5281/zenodo.2573505

[42] Team, Q.D.: AerSimulator. [Accessed February 23, 2024] (2024). https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html
[43] Eviden: Interoperability with gate-based framework. [Accessed March 20, 2024] (2024). https://myqlm.github.io/%3Amyqlm%3Ainteroperability.html
[44] Qiskit Community: Qiskit: An Open-Source Framework for Quantum Computing (2017). doi:10.5281/zenodo.2562110. https://github.com/Qiskit/qiskit
[45] Cross, A.W., Bishop, L.S., Smolin, J.A., Gambetta, J.M.: Open Quantum Assembly Language (2017). 1707.03429
[46] Smith, R.S., Curtis, M.J., Zeng, W.J.: A Practical Quantum Instruction Set Architecture (2016)
[47] Xanadu: Pennylane. [Accessed April 30, 2024] (2024). https://pennylane.ai/
[48] Staff, I.: IonQ website. [Accessed April 30, 2024] (2024). https://ionq.com/
[49] Steiger, D.S., Häner, T., Troyer, M.: ProjectQ: an open source software framework for quantum computing. Quantum **2**, 49 (2018). doi:10.22331/q-2018-01-31-49
[50] Häner, T., Steiger, D.S., Svore, K., Troyer, M.: A software methodology for compiling quantum programs. Quantum Science and Technology **3**(2), 020501 (2018). doi:10.1088/2058-9565/aaa5cc
[51] Developers, C.: Cirq. doi:10.5281/zenodo.4750446. https://doi.org/10.5281/zenodo.4750446
[52] wire, H.: Alice & Bob Partners with OVHcloud to Bring Advanced Quantum Emulation to Public Cloud Users. [Accessed February 1, 2024] (2023). https://www.hpcwire.com/off-the-wire/alice-bob-partners-with-ovhcloud-to-bring-advanced-quantum-emulation-to-public-cloud-users/
[53] for Theoretical Physics III, I., of Stuttgart, U.: Rydberg Quantum Computer Demonstrator. [Accessed April 30, 2024] (2024). https://thequantumlaend.de/qryddemo/
[54] Eviden: qat.plugins.Nnizer. [Accessed February 13, 2024] (2023). https://qlm.bull.com/qlmaas-doc/04_api_reference/module_qat/module_plugins/%3Aqlm%3Annizer.html
[55] Hirata, Y., Nakanishi, M., Yamashita, S., Nakashima, Y.: An efficient method to convert arbitrary quantum circuits to ones on a linear nearest neighbor architecture. In: 2009 Third International Conference on Quantum, Nano and Micro Technologies, pp. 26–33 (2009). doi:10.1109/ICQNM.2009.25
[56] Li, G., Ding, Y., Xie, Y.: Tackling the qubit mapping problem for nisq-era quantum devices. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. ASPLOS '19, pp. 1001–1014. Association for Computing Machinery, New York, NY, USA (2019). doi:10.1145/3297858.3304023. https://doi.org/10.1145/3297858.3304023
[57] Zulehner, A., Paler, A., Wille, R.: Efficient mapping of quantum circuits to the ibm qx architectures. In: Proceedings of the 2018 Design, Automation and Test in Europe Conference and Exhibition, pp. 1135–1138 (2018). doi:10.23919/DATE.2018.8342181
[58] Saeedi, M., Wille, R., Drechsler, R.: Synthesis of quantum circuits for linear

nearest neighbor architectures. Quantum Information Processing **10**(3), 355–377 (2010). doi:10.1007/s11128-010-0201-2

[59] Team, Q.D.: Fake Provider. [Accessed September 1, 2023] (2023). https://qiskit.org/documentation/apidoc/providers_fake_provider.html#fake-providers

[60] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: Application: adder_n4. [Accessed October 15, 2023] (2022). https://github.com/pnnl/QASMBench/tree/master/small/adder_n4

[61] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: Application: cat_state_n4. [Accessed October 15, 2023] (2022). https://github.com/pnnl/QASMBench/tree/master/small/cat_state_n4

[62] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: Application: grover_n2. [Accessed October 15, 2023] (2022). https://github.com/pnnl/QASMBench/tree/master/small/grover_n2

[63] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: Application: iswap_n2. [Accessed October 15, 2023] (2022). https://github.com/pnnl/QASMBench/tree/master/small/iswap_n2

[64] Li, A., Stein, S., Krishnamoorthy, S., Ang, J.: Application: error_correctiond3_n5. [Accessed October 15, 2023] (2022). https://github.com/pnnl/QASMBench/tree/master/small/error_correctiond3_n5

[65] To, S.H.: Percent Error / Percent Difference: Definition, Examples. [Accessed May 26, 2024] (2024). https://www.statisticshowto.com/percent-error-difference/

[66] Tilly, J., Chen, H., Cao, S., Picozzi, D., Setia, K., Li, Y., Grant, E., Wossnig, L., Rungger, I., Booth, G.H., Tennyson, J.: The variational quantum eigensolver: A review of methods and best practices. Physics Reports **986**, 1–128 (2022). doi:10.1016/j.physrep.2022.08.003

Appendix A Comparison Lindblad and Thermal Relaxation Models

To ensure the equivalence of the Lindblad noise model in Qaptiva and the Thermal Relaxation model in Qiskit, we will develop the Kraus operator matrices for both in this appendix.

A.1 Lindblad Model

The Lindblad noise model applied in Qaptiva includes both the amplitude and phase damping.

A.1.1 Amplitude damping channel (AD)

The Kraus operational elements obtained are the following:

$$E_0^{AD} = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1 - p_{AD}(t)} \end{bmatrix} \quad E_1^{AD} = \begin{bmatrix} 0 & \sqrt{p_{AD}(t)} \\ 0 & 0 \end{bmatrix}, \tag{A1}$$

where $p_{AD}(t)$ represents the evolution of the amplitude damping probability defined as:

$$p_{AD}(t) = 1 - exp\left(\frac{-t}{T_1}\right), \tag{A2}$$

with t being the time at which the amplitude damping occurs, and T_1 the characteristic decay time [34].

A.1.2 Phase damping channel (PD)

The Kraus operators of this noise model are represented as:

$$E_0^{PD} = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1 - p_{PD}(t)} \end{bmatrix} \quad E_1^{PD} = \begin{bmatrix} 0 & 0 \\ 0 & \sqrt{p_{PD}(t)} \end{bmatrix}. \tag{A3}$$

Where $p_{PD}(t)$ is the evolution of the probability and is expressed as:

$$p_{PD}(t) = 1 - exp\left(\frac{-t}{T_\phi}\right). \tag{A4}$$

In relation to this equation, an exponential decay with a characteristic decay time T_ϕ is observed. The value of T_ϕ is defined as:

$$T_\phi = \frac{2T_1T_2}{2T_1 - T_2}, \tag{A5}$$

where T_2 represents the transverse decay time and T_1 the longitudinal decay time [34].

The combination of these channels is achieved by composing both noise channels, resulting in a combined channel. To do this, a multiplication of the Kraus operators of the individual channels is carried out, and the products are detailed below.

$$E_0^C = E_0^{PD} E_0^{AD} = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-p_{AD}}\sqrt{1-p_{PD}} \end{bmatrix}, \tag{A6}$$

$$E_1^C = E_0^{PD} E_1^{AD} = \begin{bmatrix} 0 & \sqrt{p_{AD}} \\ 0 & 0 \end{bmatrix}, \tag{A7}$$

$$E_2^C = E_1^{PD} E_0^{AD} = \begin{bmatrix} 0 & 0 \\ 0 & \sqrt{1-p_{AD}}\sqrt{p_{PD}} \end{bmatrix}. \tag{A8}$$

There is a fourth possible combination, as shown below:

$$E_3^C = E_1^{PD} E_1^{AD} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}. \tag{A9}$$

However, since this channel represents a null effect, it has not been considered in the previous analysis.

By substituting the probabilities associated with amplitude damping and pure dephasing into the matrices, the following operators are obtained.

$$E_0^C = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{e^{-t/T_1}}\sqrt{e^{-t/T_\phi}} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{-t/T_2} \end{bmatrix}, \tag{A10}$$

$$E_1^C = \begin{bmatrix} 0 & \sqrt{1-e^{-t/T_1}} \\ 0 & 0 \end{bmatrix}, \tag{A11}$$

$$E_2^C = \begin{bmatrix} 0 & 0 \\ 0 & \sqrt{e^{-t/T_1}}\sqrt{1-e^{-t/T_\phi}} \end{bmatrix}. \tag{A12}$$

The quantum channel resulting from the combination of the previous Kraus operators on a single qubit is shown below:

$$\mathcal{E}_C(\rho) = \begin{bmatrix} 1-\rho_{11}e^{-t/T_1} & \rho_{01}e^{-t/T_2} \\ \rho_{01}^* e^{-t/T_2} & \rho_{11}e^{-t/T_1} \end{bmatrix}. \tag{A13}$$

A.2 Thermal Relaxation model

This model used in Qiskit, explained in Section 2.2, is developed following the condition $T_2 \leq 2T_1$. The relaxation and decoherence times, T_1 and T_2 respectively, of a quantum computer capture the dynamics of the quantum state of a single qubit due to thermalization towards an equilibrium state with the environment.

Assuming that the initial state is represented by the density matrix expressed in Equation A14.

$$\rho = \begin{bmatrix} \rho_{00} & \rho_{01} \\ \rho_{01}^* & \rho_{11} \end{bmatrix}. \tag{A14}$$

After a period of time, the quantum state is altered by idle noise. This change in the initial state due to noise can be expressed as the quantum noisy channel:

$$\mathcal{E}_T(\rho) = \begin{bmatrix} (\rho_{00}-a_0)e^{-t/T_1}+a_0 & \rho_{01}e^{-t/T_2} \\ \rho_{01}^* e^{-t/T_2} & (a_0-\rho_{11})e^{-t/T_1}+1-a_0 \end{bmatrix}, \tag{A15}$$

where a_0 represents the thermal equilibrium state, which in superconducting qubits corresponds to $a_0 = 1$ [66]. Furthermore, since matrix represented in A14 exhibits the following relation $\rho_{11} = 1 - \rho_{00}$. If we substitute these data into Equation A15, we obtain the same noisy channel as in the case of the Lindblad model, Equation A13.

The noise channel matrices representing both the Thermal Relaxation model and the Lindblad model are identical. This confirms the analogous nature of both models, highlighting their equivalence in terms of describing quantum noise.

Appendix B Nnizer plugin

The ***Nnizer*** plugin features two different configurations, the first allows the selecting the SWAP gate insertion method and offers four available algorithms:

- *atos*: the Atos algorithm presents an efficient method for converting an arbitrary quantum circuit into one adapted to a Linear Nearest Neighbor (LNN) architecture. This method achieves a small overhead and has lower time complexity than naive techniques such as exhaustive search [55].
- *sabre*: sabre algorithm, a SWAP-based bidirectional heuristic search algorithm that allows controlling parallelization in additional SWAP, generating circuits compatible with hardware, and finding a balance between circuit depth and the number of gates [56].
- *bka*: the best-known algorithm is divided into two key steps: circuit partitioning respecting the CNOT gate constraints and determining a specific mapping for each layer, thus minimizing the need for additional gates [57].
- *pbn*: the pattern-based optimizer provides a comprehensive approach that includes a post-synthesis optimization method using template matching to reduce the circuit cost in LNN architectures, an exact synthesis method for small functions with interaction between neighboring qubits, and reordering strategies to reduce the distance between non-neighboring qubits [58].

In addition, the Nnizer plugin allows the configuration of the initial qubit mapping through the following configurations:

- *annealing*: this configuration involves updating the initial mapping using simulated annealing. This method is parameterized by a score function to be maximized, maximum number of iterations performed, and initial temperature.
- *reverse*: the initial mapping is updated using the reverse traversal method described in [56]. This approach was parameterized using a scoring function that was maximized.
- *none*: the i-th qubit of the circuit is directly assigned to the i-th qubit of the hardware.

Each method introduces a different quantity and arrangement of the SWAP gates.

Appendix C Nnizer plugin examples

To illustrate the need for incorporating SWAP gates and the difference in the initial mapping, we begin by selecting a circuit with entangled qubits, as shown in Figure C1. In Figure C2, the application of the *sabre* method with an initial map *none* is observed. This method has inserted four SWAP gates into the circuit to match the ***ibm_perth*** backend topology.

The example circuit requires connection between $q1$ and $q3$, a task incompatible with the given topology. Hence, SWAP gates are incorporated to enable this connection. If the *sabre* method is chosen with the initial map set to *annealing*, it is observed that the resulting circuit is identical to the one presented in Figure C2.

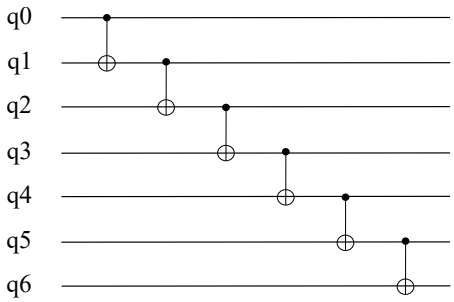

Fig. C1 Original circuit.

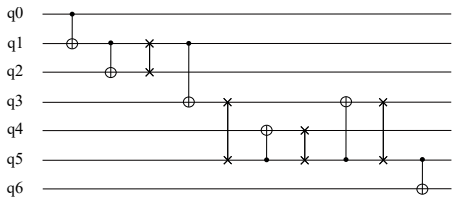

Fig. C2 Nnizer optimization (none and annealing method).

Figure C3 shows the application of the *sabre* method with an initial map *reverse* on the original circuit of Figure C1. This method has added three SWAP gates to the circuit. Upon closer analysis of the circuit, it is evident that the order of the initial qubits has been modified according to the scheme in Figure C4, where the first qubit represents the order of the original circuit, and the second represents the order after the initial mapping.

This circuit optimization results in a reduction in the number of SWAP gates, although it introduces greater complexity in the interpretation of the results. This increased complexity stems from the lack of specification regarding the initial order of qubits.

Consequently, it would be necessary to review each circuit individually and establish the relationship between the input and output qubits for the correct interpretation of the results.

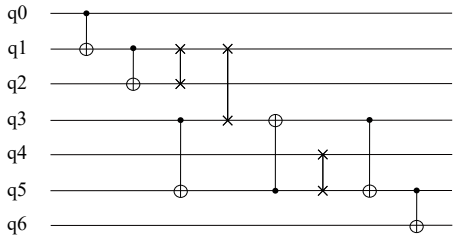

Fig. C3 Nnizer optimization (reverse method).

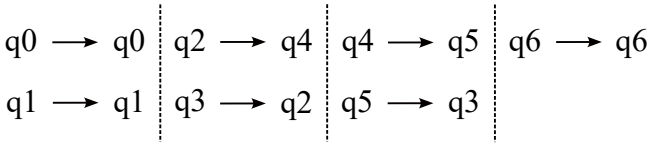

Fig. C4 Qubit scheme in the reverse method.

Table C1 shows the number of possible configurations that each evaluated circuit can acquire, as well as the result of how the transpilation process modifies the metrics of the circuits in terms of number of gates (decomposition into native gates) and the addition of SWAP gates (adaptation to the ibm_perth backend topology).

Table C1 Circuits after transpilation.

Circuit Name	Possible Configuration	1-qubit Gates	2-qubit Gates	SWAP Gates	Circuit Depth
adder_n4[60]	8	18	10	3	22
cat_state_n4[61]	8	6	3	1	8
grover_n2[62]	4	15	2	–	13
iswap_n2[63]	4	15	2	–	13
error_correctiond3_n5[64]	1	188	61	1	147

Appendix D Fidelity tables for each circuit configuration

This appendix contains fidelity tables for every possible configuration of each circuit, with data rounded to three decimal places. Each fidelity value for each configuration is an average of the values obtained over 10 iterations. These averages will be used to compare circuits and environments, as they represent the mean of all possible configurations for each circuit.

Fidelity values closest to the mean for each circuit are in blue. It is important to note that in determining which configuration is closest to the mean, all decimals have been taken into account to ensure greater precision.

For the fidelity values obtained with Qaptiva, for the *adder_n*4 circuit, the configuration that most closely approaches the mean in most of the noise models used is 010. For the *cat_state_n*4 circuit, the highlighted configuration is 001, while for *grover_n*2 it is 01. In the case of *iswap_n*2, configuration 10 is closer to the mean in the deterministic mode, while configuration 01 is closer to the stochastic mode.

For the fidelity values obtained with Qiskit, there is no predominant configuration for the *adder_n*4 circuit. For *cat_state_n*4, the configuration that most closely approaches the mean of the circuit is 010, for *grover_n*2 it is 01 and for *iswap_n*2 it is 10.

It is worth mentioning that the *error_correctiond*3_*n*5 circuit only has one configuration, so it is not necessary to calculate any averages.

Table D2 Fidelity obtained for each circuit configuration for the Qaptiva environment. (1)

Fidelity results by circuit configuration in Qaptiva environment								
		QPU	Qaptiva: Deterministic Mode			Qaptiva: Stochastic Mode		
		ibm_perth	Depolarizing Channel	Lindblad Model	Depolarizing Channel+ Lindblad Model	Depolarizing Channel	Lindblad Model	Depolarizing Channel+ Lindblad Model
Circuit	**Configuration**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**
adder_n4	000	0.698	0.711	0.985	0.702	0.711	0.985	0.703
adder_n4	001	0.620	0.735	0.969	0.713	0.735	0.969	0.712
adder_n4	010	0.618	0.712	0.947	0.675	0.711	0.946	0.676
adder_n4	011	0.601	0.704	0.958	0.676	0.704	0.958	0.676
adder_n4	100	0.663	0.723	0.938	0.679	0.723	0.938	0.678
adder_n4	101	0.657	0.704	0.945	0.668	0.704	0.945	0.667
adder_n4	110	0.711	0.710	0.878	0.694	0.710	0.976	0.694
adder_n4	111	0.708	0.734	0.957	0.704	0.734	0.957	0.704
average		0.660	0.717	0.947	0.686	0.717	0.959	0.689
cat_state_n4	000	0.854	0.858	0.989	0.849	0.858	0.989	0.849
cat_state_n4	001	0.849	0.858	0.978	0.840	0.857	0.978	0.840
cat_state_n4	010	0.849	0.858	0.986	0.847	0.858	0.985	0.846
cat_state_n4	011	0.846	0.858	0.975	0.837	0.858	0.974	0.837
cat_state_n4	100	0.846	0.858	0.940	0.850	0.858	0.989	0.849
cat_state_n4	101	0.854	0.857	0.972	0.834	0.857	0.972	0.833
cat_state_n4	110	0.849	0.857	0.987	0.847	0.857	0.987	0.847
cat_state_n4	111	0.841	0.857	0.978	0.838	0.857	0.978	0.838
average		0.848	0.858	0.975	0.843	0.858	0.982	0.842

Table D3 Fidelity obtained for each circuit configuration for the Qaptiva environment. (2)

Fidelity results by circuit configuration in Qaptiva environment								
		QPU	Qaptiva: Deterministic Mode			Qaptiva: Stochastic Mode		
		ibm_perth	Depolarizing Channel	Lindblad Model	Depolarizing Channel+ Lindblad Model	Depolarizing Channel	Lindblad Model	Depolarizing Channel+ Lindblad Model
Circuit	**Configuration**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**
grover_n2	00	0.894	0.904	1.000	0.904	0.904	1.000	0.904
grover_n2	01	0.903	0.904	1.000	0.904	0.904	1.000	0.904
grover_n2	10	0.851	0.907	1.000	0.907	0.906	1.000	0.907
grover_n2	11	0.921	0.907	1.000	0.906	0.907	1.000	0.907
average		0.892	0.905	0.999	0.905	0.905	0.999	0.905
iswap_n2	00	0.927	0.904	1.000	0.904	0.904	1.000	0.904
iswap_n2	01	0.856	0.904	1.000	0.904	0.904	1.000	0.904
iswap_n2	10	0.889	0.904	1.000	0.904	0.904	0.999	0.904
iswap_n2	11	0.893	0.904	1.000	0.904	0.903	0.999	0.904
average		0.891	0.904	1.000	0.904	0.904	1.000	0.904
error_correctiond3_n5	00000	0.581	0.375	0.500	0.500	0.375	0.500	0.500

Table D4 Fidelity obtained for each circuit configuration for the Qiskit environment. (1)

Fidelity results by circuit configuration in Qiskit environment					
		QPU	Qiskit		
		ibm_perth	Depolarizing Channel	Thermal Relaxation Model	Depolarizing Channel+ Thermal Relaxation Model
Circuit	Configuration	Fidelity	Fidelity	Fidelity	Fidelity
adder_n4	000	0.698	0.810	0.967	0.784
adder_n4	001	0.620	0.829	0.942	0.772
adder_n4	010	0.618	0.798	0.915	0.725
adder_n4	011	0.601	0.803	0.921	0.735
adder_n4	100	0.663	0.811	0.908	0.721
adder_n4	101	0.657	0.800	0.914	0.725
adder_n4	110	0.711	0.800	0.943	0.749
adder_n4	111	0.708	0.826	0.914	0.749
average		0.660	0.810	0.928	0.745
cat_state_n4	000	0.854	0.934	0.983	0.909
cat_state_n4	001	0.849	0.934	0.983	0.910
cat_state_n4	010	0.849	0.934	0.983	0.909
cat_state_n4	011	0.846	0.933	0.983	0.908
cat_state_n4	100	0.846	0.934	0.984	0.911
cat_state_n4	101	0.854	0.932	0.977	0.901
cat_state_n4	110	0.849	0.934	0.984	0.910
cat_state_n4	111	0.841	0.934	0.984	0.911
average		0.848	0.934	0.983	0.909

Table D5 Fidelity obtained for each circuit configuration for the Qiskit environment. (2)

Fidelity results by circuit configuration in Qiskit environment					
		QPU	Qiskit		
		ibm_perth	Depolarizing Channel	Thermal Relaxation Model	Depolarizing Channel+ Thermal Relaxation Model
Circuit	**Configuration**	**Fidelity**	**Fidelity**	**Fidelity**	**Fidelity**
grover_n2	00	0.894	0.958	0.980	0.932
grover_n2	01	0.903	0.959	0.986	0.940
grover_n2	10	0.851	0.961	0.985	0.939
grover_n2	11	0.921	0.961	0.990	0.952
average		0.892	0.960	0.985	0.941
iswap_n2	00	0.927	0.959	0.993	0.952
iswap_n2	01	0.856	0.959	0.989	0.944
iswap_n2	10	0.889	0.959	0.985	0.942
iswap_n2	11	0.893	0.958	0.979	0.932
average		0.891	0.959	0.986	0.942
error_correctiond3_n5	00000	0.588	1.000	1.000	1.000